

Estimating the Number of Nodes and Paths in a DAG by Following a Random Path

Günter Rote  

Institut für Informatik, Freie Universität Berlin

Abstract

We describe a simple method to compute an unbiased estimate of the number of nodes of a directed acyclic graph (DAG) by following a random directed path. It generalizes in a straightforward way a method of D. E. Knuth from the 1960s for estimating the size of a rooted tree. A slight variation allows to estimate the number of source-sink paths.

We extend the algorithm to use many “agents” that walk cooperatively through the graph. This extension unifies and generalizes several methods that have been proposed for approximately counting the nodes of a tree.

2012 ACM Subject Classification Mathematics of computing → Graph algorithms

Keywords and phrases Approximate counting, random walk, importance sampling, polytopes

Contents

1	Problem statement and the Path Sampling algorithm (Algorithm P)	2
1.1	Weighted selection	3
1.2	No backward proportions: counting paths	4
1.3	Analysis of the expectation	4
2	Experiments	5
2.1	Permutations	6
2.2	Cubes and Klee-Minty cubes	7
3	Computing the variance of X	8
4	Importance Sampling	10
5	Employing a team of agents	11
5.1	The Team Sampling family of algorithms	11
5.2	Bucket Sampling (Algorithm BS)	12
5.3	Crowd Sampling (Algorithm CS)	14
5.4	Correctness of the Team Sampling scheme	14
5.5	Generalized Condensation	17
5.6	The variance for Team Sampling strategies	18
5.7	Relation to dynamic programming	18
6	Conclusion	19
6.1	Provable approximation quality?	19
6.2	Extension to arc weights	20
6.3	Some applications	20
6.3.1	Vertices of a polytope	20
6.3.2	Polyominoes	20
6.3.3	Triangulations of a planar point set	21
6.3.4	Linear extensions of a two-dimensional partial order	21
A	Importance Sampling for Klee-Minty cubes	23
A.1	Importance Sampling by the size of the reachable set	23
B	Details about implementing Klee-Minty cubes	25
C	Purdum’s partial backtracking method	25

1 Problem statement and the Path Sampling algorithm (Algorithm P)

We are given a directed acyclic graph (DAG) $G = (V, E)$ and a *cost function* $c: V \rightarrow \mathbb{R}$ on the nodes. The task is to estimate the total cost $c(G) := \sum_{v \in V} c(v)$.

The intended setting of the problem is that G is too large to be fully scanned, but we have access to the neighbors of a node and can thereby explore parts of G . In particular, for the basic Path Sampling algorithm, we make the following assumptions:

- There is a single *source* node s_0 from which all other nodes can be reached by directed paths.
- For any node, we can pick a *successor* node (i. e., a neighbor via an outgoing arc) uniformly at random.
- For any node v , we can determine its outdegree $d^+(v)$, i. e., the number of its *successors*, its indegree $d^-(v)$, i. e., the number of its *predecessors*, and we can access its cost $c(v)$.

The following algorithm computes an unbiased estimate X of $c(G)$ by walking along a directed path from the source. Figure 1 illustrates the algorithm on a small example.

Algorithm P: Path Sampling

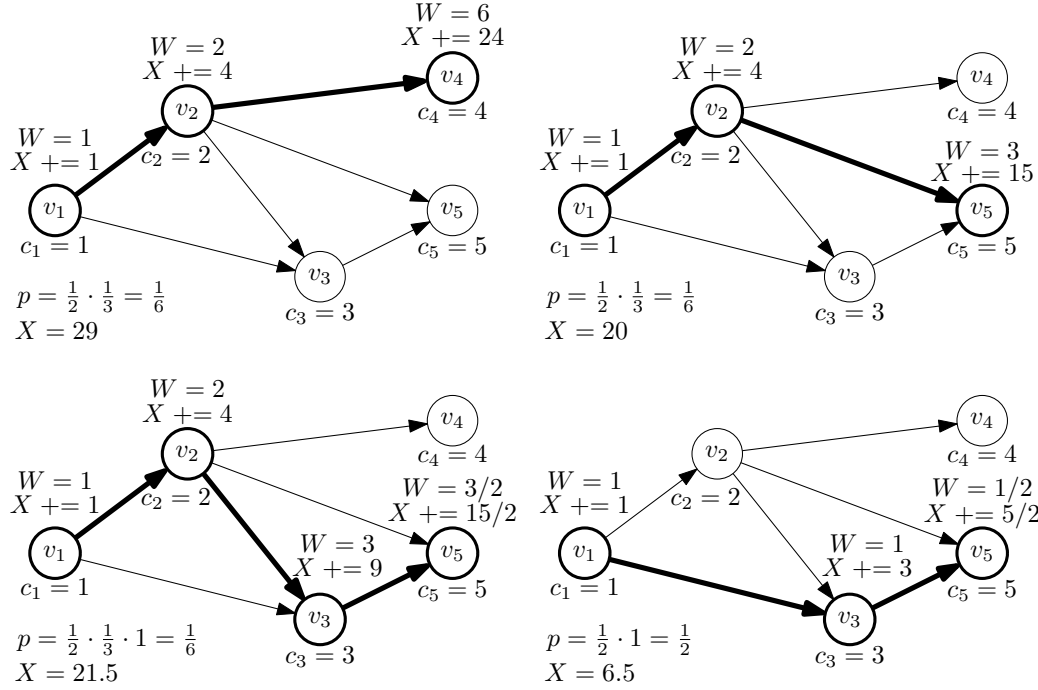
```
// Initialization:
X := 0 // accumulated estimate
u := s0 // current node, starts at the source
W := 1 // weight
loop
  // visit u:
  X := X + W · c(u) // augment the estimate
  if u has no successor, terminate and return X as the estimate
  // otherwise, proceed to a random successor v:
  (R) pick a node v among the d+(u) successors of u uniformly at random
  (U) W := W × d+(u) / d−(v) // update the weight
      u := v // go to v
```

Path sampling for trees. Algorithm P was originally developed for rooted trees by Don Knuth in 1962. It was first mentioned in a paper of Hall and Knuth [15] in 1965, and it has been investigated in its own right in [17], see also [18, Sect. 7.2.2, pp. 46–51, Corollary E]. For a tree, the indegree $d^-(v)$ is always 1, and the weight W is just the product of the outdegrees of the visited nodes.

Our algorithm is a straightforward extension to DAGs. To compensate for the fact that there can be several ways to reach a given node v , we divide W by the indegree $d^-(v)$ when going to v .

Node costs. The original purpose and the most wide-spread application of Algorithm P is to estimate the cost of backtracking or branch-and-bound algorithms. The backtracking tree or branch-and-bound tree that such an algorithm traverses may be enormous, while at other times, it is of moderate size and the algorithm terminates reasonably fast. If such a program has been running for a while, it would be useful to know whether it is just about to finish or it would continue running for years. Therefore it is desirable to have a good prediction of the remaining running time. In these applications the cost $c(v)$ is the time to process node v .

In other applications, the nodes represent some combinatorial objects, and one wishes to determine the number of these objects. In these situations, one uses uniform costs $c(v) = 1$



54 **Figure 1** A DAG G with five nodes $v_1, \dots, v_5$. The costs are $c_i = c(v_i) = i$, for a total cost of
 55 $c(G) = 1 + 2 + 3 + 4 + 5 = 15$. There are four sample paths that Algorithm P can take. The expected
 56 estimate is $E[X] = \frac{1}{6} \times 29 + \frac{1}{6} \times 20 + \frac{1}{6} \times 21.5 + \frac{1}{6} \times 6.5 = \frac{29+20+21.5+19.5}{6} = \frac{90}{6} = 15$.

74 for all nodes v . Avis and Devroye used the tree version of Algorithm P to estimate the
 75 number of vertices of a polytope [3]. If one wishes to count only the nodes with certain
 76 characteristics, this can be arranged by choosing $c(v) \in \{0, 1\}$. With appropriate costs, one
 77 can gather statistics about various properties of these objects, and of course, one can easily
 78 track several such estimates simultaneously.

79 1.1 Weighted selection

80 In line (R), the random successor does not have to be selected uniformly, but we can choose
 81 arbitrary positive transition probabilities $\vec{p}_{uv}$ subject to the constraint that they form a
 82 probability distribution:

$$83 \quad \sum_v \vec{p}_{uv} = 1, \text{ for all } u \text{ with } d^+(u) > 0 \quad (1)$$

84 We call the quantities $\vec{p}_{uv}$ the *forward proportions*. (The arrow above the p does not signify
 85 a vector, but it reminds us of the direction in which $\vec{p}_{uv}$ operates.)

86 Similarly, the incoming edges at a node v need not be weighted equally, but they can be
 87 weighted by quantities $\tilde{q}_{uv}$ that sum to 1 for the incoming arcs of each node v :

$$88 \quad \sum_u \tilde{q}_{uv} = 1, \text{ for all } v \neq s_0 \quad (2)$$

89 We call the quantities $\tilde{q}_{uv}$ the *backward proportions*. (They don't have a probabilistic
 90 interpretation, and they can even be negative.)

91 The update of the weight when traversing the arc uv in line (U) is replaced by

$$92 \quad (U_q) \quad W := W \times \tilde{q}_{uv} / \vec{p}_{uv}.$$

93 The standard choices are $\vec{p}_{uv} = 1/d^+(u)$ and $\tilde{q}_{uv} = 1/d^-(v)$.

94 ► **Theorem 1.** *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying (1) and backward*
 95 *proportions $\tilde{q}_{uv}$ satisfying (2), Algorithm P produces an unbiased estimate of $c(G)$, i. e.,*

$$96 \quad E[X] = c(G).$$

97 We will prove this below as a consequence of a more general statement.

98 1.2 No backward proportions: counting paths

99 If we omit the backward proportions, we get an estimate of the *number of paths starting*
 100 *from the source.* The weight updated step (U) is replaced by

$$101 \quad (\text{U}_1) \quad W := W / \vec{p}_{uv}.$$

102 ► **Theorem 2.** *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying (1), Algorithm P*
 103 *with update step (U₁) produces an estimate X whose expected value is*

$$104 \quad E[X] = \sum_{v \in V} c(v) \cdot \#(\text{paths from } s_0 \text{ to } v \text{ in } G).$$

105 As a special case, we can set $c(v) = 1$ if v is a sink and $c(v) = 0$ otherwise, and we get the
 106 number of source-sink paths.

107 More generally, we can replace the backward proportions by arbitrary real arc weights
 108 $e_{uv} \in \mathbb{R}$. The weight function is extended from arcs to paths in the usual way by defining
 109 $e(P)$ as the product of the arc weights along the path P :

$$110 \quad e(P) = e(u_0 u_1 \dots u_k) = e_{u_0 u_1} e_{u_1 u_2} \dots e_{u_{k-1} u_k}$$

111 ► **Theorem 3.** *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying (1) and arbitrary*
 112 *arc weights e_{uv} , Algorithm P with update step*

$$113 \quad (\text{U}_e) \quad W := W \times e_{uv} / \vec{p}_{uv}$$

114 *produces an estimate X whose expected value is the quantity $c_e(G)$ defined as follows:*

$$115 \quad E[X] = c_e(G) := \sum_{v \in V} \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) c(v)$$

116 1.3 Analysis of the expectation

117 **Proof of Theorem 3.** We define for every node v the random variable

$$118 \quad W_v := \begin{cases} \text{the value of } W \text{ at the time that } v \text{ was visited,} & \text{if } v \text{ was visited} \\ 0, & \text{if } v \text{ was not visited} \end{cases}$$

119 Then $X = \sum_{v \in V} W_v \cdot c(v)$, and therefore, $E[X] = \sum_{v \in V} E[W_v] \cdot c(v)$, by linearity of
 120 expectation. The theorem is a direct consequence of the following lemma.

121 ► **Lemma 4.** *For every node v , $E[W_v] = \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P)$.*

122 **Proof.** By definition, the expectation of W_v can be written as follows:

$$123 \quad E[W_v] = \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} \Pr[\text{path } P \text{ is taken}] \times (W_v \text{ as a result of path } P) \quad (3)$$

124 For a path $P = u_0 u_1 \dots u_k$,

$$125 \quad \Pr[\text{path } P \text{ is taken}] = \frac{1}{\vec{p}_{u_0 u_1} \vec{p}_{u_1 u_2} \dots \vec{p}_{u_{k-1} u_k}}.$$

126 If the path P is taken, then

$$127 \quad W_v = \frac{e_{u_0 u_1}}{\vec{p}_{u_0 u_1}} \cdot \frac{e_{u_1 u_2}}{\vec{p}_{u_1 u_2}} \dots \frac{e_{u_{k-1} u_k}}{\vec{p}_{u_{k-1} u_k}}.$$

128 Thus, in each product term in (3), the $\vec{p}$'s cancel, and the claim of the lemma follows. The
129 proof of Theorem 3 is thus also complete. ◀

130 Theorem 2 is just a special case of Theorem 3 with uniform arc weights $e \equiv 1$.

131 **Proof of Theorem 1 from Theorem 3.** Here the weights are the backward proportions
132 $e_{uv} = \tilde{q}_{uv}$. We only need to show the following statement:

133 ▶ **Lemma 5.** *For every node v ,*

$$134 \quad \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) = 1.$$

135 **Proof.** This is easy to prove by induction on the length of the longest path from s_0 to v .

136 For the source $v = s_0$, the claim follows directly from the fact that the degenerate path
137 $P = s_0$ with no arcs has weight 1.

138 For a node $v \neq s_0$, let $u_1, \dots, u_d$ be the predecessor nodes of v , with $d = d^-(v)$. Then

$$\begin{aligned} 139 \quad \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) &= \sum_{i=1}^d \left(\sum_{\text{paths } P' \text{ from } s_0 \text{ to } u_i} e(P') \tilde{q}_{u_i v} \right) \\ 140 \quad &= \sum_{i=1}^d \left(\sum_{\text{paths } P' \text{ from } s_0 \text{ to } u_i} e(P') \right) \tilde{q}_{u_i v} = \sum_{i=1}^d \tilde{q}_{u_i v} = 1, \end{aligned}$$

141 since each parenthesizes expression in the last line is 1, by the induction hypothesis, and the
142 backward proportions sum to 1, by assumption (2). ◀

143 2 Experiments

145 To test the accuracy of the estimation, we tried the algorithm on two families of DAGs whose
146 size we know. The reader who is more interested in the theoretical properties of the estimate
147 is encouraged to skip the experimental sections with the tables. The PYTHON programs by
148 which these preliminary experiments were conducted are available in a public repository¹,
149 together with tables of the results of more extensive experiments.

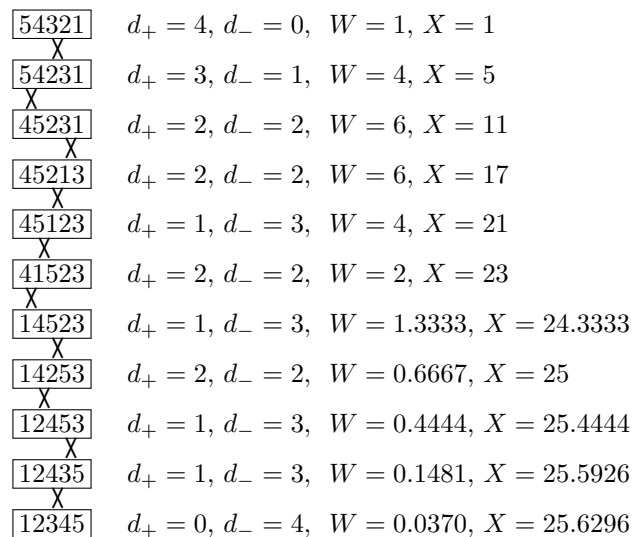
150 In our experiments, we have so far investigated only the basic version of the algorithm
151 for node counting, and we have worked exclusively with uniform costs $c(v) = 1$.

152 Some more interesting potential applications of the method are mentioned in Section 6.3.

144 ¹ <https://github.com/guenterrote/estimate-DAG>

153 **2.1 Permutations**

154 The nodes of the *permutation DAG* are the $n!$ permutations of the elements $\{1, 2, \dots, n\}$.
 155 The source node is the decreasing permutation $n, n-1, \dots, 3, 2, 1$, and the successors of a
 156 permutation are the permutations that can be reached by swapping two adjacent elements
 157 that are out of order, as in bubblesort. The sink is the sorted permutation $123 \dots n$. Figure 2
 158 shows a random path from the source 54321 to the sink 12345 for $n = 5$, leading to the
 159 estimate $X \approx 25.6296$. (The true size is $5! = 120$.)



160 ■ **Figure 2** A path in the permutation DAG

161 We have tested the algorithm for the permutation DAG for $n = 8$ and $n = 9$. Table 1
 162 displays the average of up to $m = 10^7$ repetitions of Algorithm P. The true value $n!$
 163 is displayed in the last row. We see that the values gradually converge to an estimate that is in
 164 the right ballpark.

165 We also show the empirical standard deviation (the square root of the empirical variance)
 166 of the first m samples, as well as value of the largest sample. The last row gives the precise
 167 theoretical value of the standard deviation of the random variable X , computed by the
 168 method of Section 3.

169 We see that the standard deviation is rather high, at least one order of magnitude higher
 170 than the mean. We also see that the largest estimate is very much larger than the mean.
 171 After $m = 1000$ estimates for $n = 8$, we have seen an estimate of around 4.83×10^7 , leading to
 172 the outlier mean of 77,136. These phenomena indicate that the estimate is unreliable. Indeed,
 173 in the context of backtracking trees, it has often been observed that, while the estimate
 174 produced by Path Sampling is unbiased, it has a high uncertainty, and the information it
 175 gives is sometimes useless.

176 For such situations, Chatterjee and Diaconis [8, Section 2] proposed to look at the quantity
 177 $\hat{\chi} = \frac{\max\{X_1, \dots, X_m\}}{X_1 + \dots + X_m}$, see also [18, Eq. (34), p. 50]. A value of $\hat{\chi}$ close to 1 indicates that the
 178 mean is essentially determined by a single sample that dominates the other samples by far.
 179 Chatterjee and Diaconis argue that a reasonable policy is to trust the experiments when $\hat{\chi}$ is
 180 small. Our tables show the Chatterjee–Diaconis score $\hat{\chi}$ in the last column.

Permutations						
m	$n = 8$			$n = 9$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$39,331.8 \pm 59,061.9$	1.42×10^5	0.36044	$48,168.7 \pm 56,348.3$	1.68×10^5	0.34803
10^2	$28,772.5 \pm 64,855.0$	3.42×10^5	0.11875	$134,918.1 \pm 4.87 \times 10^5$	4.21×10^6	0.31195
10^3	$77,136.0 \pm 1.55 \times 10^6$	4.83×10^7	0.62640	$188,542.8 \pm 1.33 \times 10^6$	2.93×10^7	0.15514
10^4	$44,629.3 \pm 6.57 \times 10^5$	4.83×10^7	0.10826	$221,679.5 \pm 2.23 \times 10^6$	1.49×10^8	0.06720
10^5	$42,382.8 \pm 1.13 \times 10^6$	2.89×10^8	0.06821	$372,754.8 \pm 1.75 \times 10^7$	4.94×10^9	0.13250
10^6	$39,837.5 \pm 8.32 \times 10^5$	3.63×10^8	0.00912	$347,645.5 \pm 1.23 \times 10^7$	4.98×10^9	0.01433
10^7	$40,820.6 \pm 1.92 \times 10^6$	5.16×10^9	0.01265	$351,350.7 \pm 2.59 \times 10^7$	4.64×10^{10}	0.01320
	40,320 $\pm 1.642 \times 10^6$	#visited = 29		362,880 $\pm 1.614 \times 10^8$	#visited = 37	

Table 1 Algorithm P for the permutation DAG. The number of visited nodes is always $1 + \binom{n}{2}$, equaling the number of layers, because all source-sink path have the same length.

2.2 Cubes and Klee-Minty cubes

We take the graph (i.e., the edge skeleton) of the unit cube in n dimensions and orient its edges by some generic objective function. It turns out that, on these instances, Algorithm P works impeccably! At distance d from the source, all nodes have indegree d and outdegree $n - d$. Hence the accumulated contributions to X are $1 + n + \binom{n}{2} + \binom{n}{3} + \dots + \binom{n}{d} + \dots = 2^n$. We always get the correct answer.

Since the behavior of Algorithm P for standard cubes is so uninteresting, we apply the algorithm to *Klee-Minty cubes* instead. Klee and Minty, in 1972, defined a perturbation of the unit n -cube for which they could show that the simplex algorithm, with certain pivot rules, visits all 2^n vertices and thus takes exponential time [16].

Figure 3 shows a version of this polytope with integral vertices, inscribed in a box with exponentially increasing side lengths rather than a cube. The arcs are oriented in the direction of the last coordinate x_n . This orientation of the arcs agrees with the original Klee-Minty cubes. Our version of the Klee-Minty cube in n dimensions is defined by the following linear program:

Maximize x_n

subject to the following $2n$ constraints:

$$\begin{aligned} 0 &\leq x_1 \leq 1 \\ x_{i-1} &\leq x_i \leq 2^i - 1 - x_{i-1}, \text{ for } i = 2, \dots, n \end{aligned} \tag{4}$$

We see that the n -dimensional Klee-Minty “box” has a *floor* facet and a *ceiling* facet, where x_n is at its lower bound $x_n = x_{n-1}$ or at its upper bound $x_n = 2^n - 1 - x_{n-1}$, respectively. Both the floor and the ceiling is an affine image of an $(n - 1)$ -dimensional Klee-Minty box. The floor has the original orientations, and in the ceiling, all orientations are reversed.

Algorithm P essentially performs the simplex algorithm with the *random-edge* pivot rule. For the expected number of steps of this pivot rule on Klee-Minty cubes, there is an easy upper bound of $O(n^2)$ [14]. A matching lower bound of $\Omega(n^2)$ is known only for a random start vertex [4].

Table 2 shows the results of two runs, with $n = 10$ and $n = 12$ dimensions. In comparison to the permutation DAG, we see that the estimate is less accurate, even though the cubes

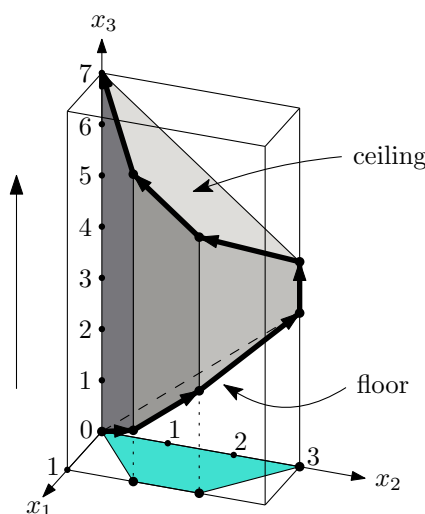


Figure 3 An integral Klee-Minty cube in $n = 3$ dimensions with the directed path through all 2^n vertices. The edges are oriented upwards in the vertical x_3 -direction.

Klee-Minty cubes							
m	$n = 10$				$n = 12$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$		mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$882.1 \pm 2,154.3$	6.96×10^3	0.78904		$1,046.4 \pm 2,134.4$	6.81×10^3	0.65084
10^2	181.2 ± 717.4	6.96×10^3	0.38409		$478.4 \pm 1,484.8$	1.21×10^4	0.25341
10^3	$961.4 \pm 11,280.5$	2.85×10^5	0.29669		$1,787.6 \pm 24,370.0$	6.97×10^5	0.38967
10^4	$711.4 \pm 18,195.4$	1.69×10^6	0.23742		$1,118.3 \pm 16,573.2$	1.05×10^6	0.09420
10^5	$1,146.6 \pm 1.42 \times 10^5$	4.42×10^7	0.38589		$1,499.9 \pm 36,213.5$	5.76×10^6	0.03842
10^6	$854.8 \pm 65,844.1$	4.42×10^7	0.05176		$2,649.3 \pm 3.50 \times 10^5$	2.71×10^8	0.10227
10^7	$866.6 \pm 85,253.1$	1.02×10^8	0.01178		$3,542.4 \pm 2.27 \times 10^6$	6.58×10^9	0.18583
	$1,024 \pm 3.246 \times 10^9$				$4,096 \pm 1.034 \times 10^{19}$		
	#visited: avg = 15.7, max = 78				#visited: avg = 20.6, max = 94		

Table 2 Algorithm P for Klee-Minty cubes

have much fewer nodes. The theoretical standard deviation is very large (around 10^{19} for $n = 12$), but this high standard deviation is not observed empirically.²

Appendix B gives some details about the representation that we used for the vertices of the Klee-Minty cubes.

3 Computing the variance of X

For test DAGs that are small enough so that one can iterate through all nodes, it is possible to compute the variance of the random estimate X for Algorithm P exactly.

We use the following basic result about a mixture of distributions:

² This extreme clash between the predicted and the observed standard deviation raises the suspicion of an error in the formulas or a programming error. However, for small values of n , the empirical standard deviation converges very well to the predicted standard deviation.

246 ► **Lemma 6** (Mixture of distributions). *Consider d random variables $Z_1, \dots, Z_d$ with means*
 247 *μ_i and variances var_i . Let Z be the mixture of these variables with proportions $\alpha_1, \dots, \alpha_d$,*
 248 *that is, Z is set to Z_i with probability α_i , where $\sum_{i=1}^d \alpha_i = 1$. Then Z has mean*

$$249 \quad \mu = \sum_{i=1}^d \alpha_i \mu_i$$

250 and variance

$$251 \quad \text{var} = \sum_{i=1}^d \alpha_i (\text{var}_i + (\mu_i - \mu)^2) = \sum_{i=1}^d \alpha_i \text{var}_i + \sum_{1 \leq i < j \leq d} \alpha_i \alpha_j (\mu_i - \mu_j)^2. \quad (5)$$

252 These equations can be seen as instances of the *law of total expectation* $E[Z] = E[E[Z|Y]]$
 253 and the *law of total variance* $\text{var}(Z) = E[\text{var}(Z|Y)] + \text{var}(E[Z|Y])$, where $Y \in \{1, \dots, d\}$
 254 is a random variable that determines which Z_i is chosen. The last expression in (5) is an
 255 alternative formula, which is sometimes more convenient for hand calculation when d is
 256 small.

257 Let us apply this to Algorithm P. For each node u , we define the random variable $D_u(W)$
 258 to be the value of all future additions to X , assuming that u is entered with weight W . Since
 259 the operations on W consist of multiplications by random quantities that are independent
 260 of W , it is clear that $D_u(W)$ is a linear function of W , i. e.,

$$261 \quad D_u(W) = F_u W,$$

262 for some random factor F_u that depends only on the graph G , together with the forward
 263 proportions $\vec{p}_{uv}$ and the arc weights.

264 We now show how the mean and the variance of the variables F_u can be computed
 265 in reverse direction of the arcs, proceeding from the end of the DAG towards the source,
 266 Eventually we arrive at the final estimate $X = F_{s_0}$, and $\text{var}(F_{s_0})$ is the quantity that we are
 267 looking for.

268 Let u be a node with d successors $v_1, \dots, v_d$. Then F_u is $c(u)$ plus a mixture of the d
 269 random variables $e_{uv_j} / \vec{p}_{uv_j} \cdot F_{v_j}$, each taken with probability $\alpha_j = \vec{p}_{uv_j}$. Let μ_j and var_j
 270 denote the mean and variance of F_{v_j} . Then, by Lemma 6, the mixture has mean

$$271 \quad \mu_0 = \sum_{j=1}^d \vec{p}_{uv_j} \cdot \frac{e_{uv_j}}{\vec{p}_{uv_j}} \cdot \mu_j = \sum_{j=1}^d e_{uv_j} \mu_j$$

272 and variance

$$273 \quad \text{var}(F_u) = \sum_{j=1}^d \vec{p}_{uv_j} \cdot \left[\left(\frac{e_{uv_j}}{\vec{p}_{uv_j}} \right)^2 \cdot \text{var}_j + \left(\frac{e_{uv_j}}{\vec{p}_{uv_j}} \mu_j - \mu_0 \right)^2 \right]$$

$$274 \quad = \sum_{j=1}^d \frac{1}{\vec{p}_{uv_j}} \left[e_{uv_j}^2 \text{var}_j + (e_{uv_j} \mu_j - \vec{p}_{uv_j} \mu_0)^2 \right]$$

275 To get the expectation of F_u , we must not forget to add the cost $c(u)$:

$$276 \quad E[F_u] = c(u) + \mu_0 = c(u) + \sum_{j=1}^d e_{uv_j} \mu_j \quad (6)$$

277 For a node u with no successors, the formulas specialize to $\text{var}(F_u) = 0$ and $E[F_u] = c(u)$.

For the special case of uniform forward proportions $\vec{p}_{uv_j} = 1/d$, we get the following formulas:

$$\text{var}(F_u) = d \cdot \sum_{j=1}^d \left[e_{uv_j}^2 \text{var}_j + \left(e_{uv_j} \mu_j - \frac{\mu_0}{d} \right)^2 \right] = d \cdot \sum_{j=1}^d e_{uv_j}^2 \text{var}_j + \sum_{1 \leq i < j \leq d} (e_{uv_i} \mu_i - e_{uv_j} \mu_j)^2$$

4 Importance Sampling

How can we use the freedom to pick the successor according to an arbitrary distribution in order to get a better estimate? For a rooted tree, it is known what the ideal forward proportions $\vec{p}_{uv}$ are: They should be proportional to the cost of the subtrees rooted at the children v of node u . If the forward proportions are chosen in this way, the estimate X will always deliver the correct answer.

In practice, one can choose the probability $\vec{p}_{uv}$ of going from u to v based on the properties of v in the graph, e.g., the number of successors or predecessors, or on characteristics of the combinatorial objects that the node v represents. We are entering the realm of heuristic arguments, and the success of any particular rule depends on the type of problem.

We have tried the simple rule to choose the probabilities proportional to the outdegrees of the successor nodes v . (If the outdegree is 0, we have taken 1 instead, to avoid zero probabilities.) Vertices with large outdegree are more likely to reach many nodes. For the permutation DAG, the outdegree (and hence indegree) of a successor v differs by at most ± 1 from u , and hence the effect of the outdegree tends to amplify. This is why it seems to make sense to choose the outdegree as a criterion.

Permutations							
m	$n = 8$				$n = 9$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$		mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$10,564.8 \pm 11,944.9$	3.32×10^4	0.31394		$234,362.6 \pm 4.41 \times 10^5$	1.28×10^6	0.54434
10^2	$15,677.9 \pm 28,749.5$	1.85×10^5	0.11808		$315,348.3 \pm 1.03 \times 10^6$	6.34×10^6	0.20111
10^3	$47,912.4 \pm 4.33 \times 10^5$	1.23×10^7	0.25653		$281,586.0 \pm 1.10 \times 10^6$	1.40×10^7	0.04980
10^4	$39,265.0 \pm 2.24 \times 10^5$	1.23×10^7	0.03130		$312,165.1 \pm 3.48 \times 10^6$	2.36×10^8	0.07569
10^5	$40,970.8 \pm 3.82 \times 10^5$	6.63×10^7	0.01619		$411,450.9 \pm 2.05 \times 10^7$	6.05×10^9	0.14698
10^6	$39,777.6 \pm 4.31 \times 10^5$	2.35×10^8	0.00592		$360,495.0 \pm 1.13 \times 10^7$	6.05×10^9	0.01678
10^7	$40,289.8 \pm 5.41 \times 10^5$	7.25×10^8	0.00180		$372,028.6 \pm 4.02 \times 10^7$	1.01×10^{11}	0.02708
	$40,320 \pm 6.316 \times 10^5$	#visited = 29			$362,880 \pm 4.665 \times 10^7$	#visited = 37	

Table 3 Algorithm P for permutations with Importance Sampling by outdegree

The results are shown in Table 3. We see that the theoretical standard deviation goes down by a factor between 2.6 and 3.5 in comparison with Table 1. In terms of empirical behavior, the comparison is not clear. The experiments are haunted by the occasional occurrences of rare events, such as unusually early appearances of high estimates like 6.05×10^9 and 1.01×10^{11} for $n = 9$.

We tried the same method also for Klee-Minty cubes, with similarly inconclusive results. Appendix A reports these results, as well as some further experiments of only theoretical value, where the size of the reachable node set is taken into account for the forward proportions.

In a different setting, namely the enumeration tree for pseudoline arrangements, it has been observed that the degrees of nodes and their children are strongly correlated, see [22,

Appendix E5, Fig. 14], suggesting that importance sampling should be helpful. It indicates that forward proportions chosen proportional to the outdegrees would underestimate the true size of the subtrees, and thus the influence of the outdegrees on the probabilities should be even stronger than just proportional. However, an experimental evaluation has not been done yet. The nodes in the deeper levels of these enumeration trees have typically several hundred to thousands of successors.

5 Employing a team of agents

5.1 The Team Sampling family of algorithms

Instead of making many independent repetitions of following a single path with Algorithm P, it is better to let several “tokens” or “cooperative agents” move through the graph simultaneously. *From now on, we assume that the arc weights e_{uv} are positive.*

We propose the following general scheme. The algorithm maintains a list A of *active nodes*, and for each element $v \in A$, a positive weight $W[v]$.

Algorithm TS: Team Sampling

1. [Initialize] $A = \{s_0\}$; $W[s_0] := 1$; $X := 0$.
2. While A is nonempty, perform any of the following two steps:
 - 2E. [Expand successors] Choose an arbitrary node u of A , and remove it from A .
Set $X := X + W[u] \cdot c(u)$. // Augment X
For each successor v of u :
If v is not present in A , insert v into A and clear its weight: $W[v] := 0$.
Then, increase its weight to $W[v] := W[v] + W[u]e_{uv}$.
 - 2C. [Condense] Choose an arbitrary subset S of A with at least 2 elements.
Pick a random node u from S , with probabilities proportional to the weights $W[u]$.
Replace S in A by the single element u , and set $W[u]$ to the sum of the weights in S .
3. [Terminate] In the end, return X .

The choice between Step 2E and Step 2C, and the choice of u in Step 2E or of S in Step 2C can be done according to some rule, depending on any information that has been stored or that is currently at hand, or even randomly. The weights can be taken into account. We prove in Section 5.4 that, whatever choices the algorithm makes, it returns an unbiased estimate X of the quantity $c_e(G)$ that was defined in Theorem 3.

Algorithm P is a special case of this algorithm where Step 2E is always directly followed by Step 2C for the whole set A , i.e., the successors of the current node. In contrast to Path Sampling, Algorithm TS as formulated does not allow a nonuniform selection among the successors through forward proportions $\vec{p}_{uv}$. Section 5.5 will show how a nonuniform selection can be accommodated by generalizing the condensation step 2C.

It can happen that a node v is removed from A and later reinserted. For example, in the graph of Figure 1, after initially expanding v_1 into the set $A = \{v_2, v_3\}$, the algorithm could first expand v_3 , and later v_2 , reinserting v_3 into A . This is obviously a waste of effort, and it can be avoided by choosing the nodes in Step 2E in some topological order of the DAG. In many applications, it is easy to obey this rule. For example, the permutation DAG is graded: All paths from the source to a node v have the same length. More generally, we may have some rank function $r: V \rightarrow \mathbb{Z}$ or $r: V \rightarrow \mathbb{R}$ such that $r(v) > r(u)$ for all arcs uv . When we have the graph of a polytope, oriented by some generic linear objective function f , then f can serve as a rank function for the nodes. In other instances, it may not be straightforward to

find out if a node v is reachable from a node u . In any case, the correctness of the algorithm does not depend on any proper order in which the nodes are chosen.

Keeping multiple copies of the same node. In the above formulation of Step 2E, we check the presence of a node v in A before trying to insert it, but this is actually not necessary. We can also insert a fresh copy of v into A with a separate weight $W[v]$, (which is initialized to 0 as usual). Accordingly, we have to treat A as a *multiset*. When such a duplication is detected, it is of course expedient to condense multiple copies of the same node v , as a special case of Step 2C; but again, the correctness of the algorithm does not depend on this.

The general scheme of Team Sampling offers a lot of freedom. We describe and test two strategies that specialize this scheme, *Bucket Sampling* and *Crowd Sampling*, both inspired by methods that have been proposed for trees in the literature.

5.2 Bucket Sampling (Algorithm BS)

This strategy is modeled after the algorithm of Pang C. Chen [9] from 1992, which was proposed for trees under the name *heuristic sampling*; Knuth called it *Chen subset sampling* [19, Section 7.2.2.9, Algorithm C]. Nodes with similar characteristics are lumped together in a group S that is to be condensed. The idea is that, if the groups are formed in a sensible way, any element $u \in S$ is as good a representative of S as any other element.

Formally, the algorithm uses a *heuristic function* $h: V \rightarrow U$ from the nodes V to some universe U , which is used to index the *buckets* to which the nodes are assigned. Ideally, the criterion by which nodes are grouped would be that the nodes have similar subtrees (when G is a tree) or similar sets of successors (when G is a DAG). However, these characteristics are usually not available or expensive to compute, so one has to settle for some easier “heuristic” characteristics. In our experiments, we have chosen the outdegree as the heuristic function.

In the original algorithm of Chen [9], a node and its ancestor node cannot be in one bucket, and what we call a bucket is called a *stratum*. This requirement, together with a proper order of processing the strata, ensures that the algorithm visits at most one node per stratum. With our relaxed notion of heuristic function, the stratum terminology is less appropriate, and we have chosen the term bucket instead.

Depending on the character of the universe U , the list of buckets can be organized in an array, a hash table, or a priority queue.

Conceptually, each bucket could store a list containing the (multi-)set of nodes v with the same values $h(v)$. The algorithm repeatedly chooses some nonempty bucket, condenses it, and expands the chosen representative.

However, the following considerations, which were already made for the original heuristic sampling method of [9], show that one only needs to store at most one node per bucket. Suppose we condense a set $S = \{u_1, u_2, u_3\}$, resulting in a representative $u' \in S$, and later condense the set $\{u', u_4, u_5, u_6\}$. It can be easily checked that the total effect is the same as condensing the set $\{u_1, u_2, u_3, u_4, u_5, u_6\}$ in one go. Thus, if we know that u_1, u_2, u_3 will eventually be condensed (possibly together with more elements that are not yet known), it makes sense to condense them right away, in order to save space.

Therefore, we store at most one node per bucket. In the expansion step, we generate successor nodes and insert them into their buckets. Whenever a node is inserted into a bucket that is already occupied, we immediately condense the set of these two nodes. (This procedure corresponds to the *reservoir sampling* method of selecting a sample from a data stream.)

The resulting algorithm with the modified main loop is shown here:

Algorithm BS: Bucket Sampling

1. [Initialize] $A = \{s_0\}$; $W[s_0] := 1$; $X := 0$.
2. While A is nonempty, perform the following step:
 - BS-2E. [Expand successors] Choose an arbitrary node u of A , and remove it from A .
 Set $X := X + W[u] \cdot c(u)$. // Augment X
 For each successor v of u :
 If A contains no node v' with $h(v) = h(v')$,
 Insert v into A and set $W[v] := W[u]e_{uv}$.
 Otherwise, i.e., if A contains a node v' with $h(v) = h(v')$,
 [Condense v and v'] Compute $\bar{W} := W[v'] + W[u]e_{uv}$.
 With probability $W[v']/\bar{W}$, set $W[v'] := \bar{W}$ (and ignore v).
 With the complementary probability, replace v' by v in A , and set $W[v] := \bar{W}$.
3. [Terminate] In the end, return X .

Experiments. We have tried this approach for the permutation DAG and the Klee-Minty cubes. As a heuristic function, we have taken the outdegree. In each step, we have selected the nonempty bucket with the largest outdegree and expanded the node stored in this bucket. The rationale behind this choice is that nodes closer to the root tend to have larger outdegree. For the permutation DAG, it would have been natural to use the level (rank) as additional information, but we have ignored this for simplicity.

The results are shown in Tables 4 and 5. The number of iterations (reported as “#visited”) is larger than for Path Sampling, and in addition, each visit incurs more work than for Path Sampling; therefore we went only up to $m = 10^6$ repetitions. One can see that Bucket Sampling with our very simple bucketing strategy improves the estimate. The standard deviation is reduced to a reasonable scale, in particular for the smaller instances.

We also report as “#generated” the total number of successor nodes that are generated and inserted into buckets; this quantity is representative of the total work of the algorithm. Our digraphs are regular graphs, of degree $n - 1$ for the permutation DAGs, and of degree n for the Klee-Minty cubes. On average, half of the incident edges are outgoing edges. Thus the number of generated nodes should be roughly the number of visited nodes times $(n - 1)/2$ or $n/2$, respectively. The actual number of generated nodes (“#generated”) is just slightly smaller than what one would expect from this calculation.

Permutations						
m	$n = 8$			$n = 9$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$34,023.3 \pm 41,991.4$	1.48×10^5	0.43367	$186,271.5 \pm 1.09 \times 10^5$	3.38×10^5	0.18146
10^2	$47,540.9 \pm 69,429.7$	4.77×10^5	0.10031	$277,473.9 \pm 4.23 \times 10^5$	3.41×10^6	0.12283
10^3	$39,992.2 \pm 47,453.0$	4.77×10^5	0.01192	$333,151.4 \pm 5.89 \times 10^5$	7.63×10^6	0.02289
10^4	$39,758.2 \pm 54,078.2$	1.14×10^6	0.00286	$362,520.5 \pm 1.09 \times 10^6$	7.77×10^7	0.02143
10^5	$40,183.6 \pm 59,067.6$	2.87×10^6	0.00071	$361,473.8 \pm 1.01 \times 10^6$	1.17×10^8	0.00324
10^6	$40,336.0 \pm 61,807.6$	5.89×10^6	0.00015	$363,775.1 \pm 9.37 \times 10^5$	1.71×10^8	0.00047
	40,320			362,880		
	#visited: avg = 74.3, max = 95			#visited: avg = 109.6, max = 144		
	#generated: avg = 247.1			#generated: avg = 414.9		

Table 4 Bucket Sampling for the permutation DAG.

Klee–Minty cubes							
m	$n = 10$				$n = 12$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$		mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$1,183.2 \pm 1,423.3$	4.64×10^3	0.39191		$2,930.8 \pm 2,237.0$	7.84×10^3	0.26759
10^2	$1,171.8 \pm 2,521.9$	1.91×10^4	0.16258		$3,486.7 \pm 5,502.1$	3.70×10^4	0.10600
10^3	$1,111.9 \pm 3,284.9$	7.72×10^4	0.06940		$3,711.8 \pm 13,201.9$	2.79×10^5	0.07510
10^4	$1,022.8 \pm 3,526.0$	2.42×10^5	0.02371		$3,964.3 \pm 18,223.7$	1.15×10^6	0.02899
10^5	$1,014.0 \pm 3,282.6$	3.22×10^5	0.00318		$3,993.5 \pm 27,014.1$	4.98×10^6	0.01246
10^6	$1,031.1 \pm 4,959.5$	2.97×10^6	0.00288		$4,104.4 \pm 44,370.2$	2.05×10^7	0.00500
	1,024				4,096		
	#visited: avg = 193.2, max = 1328				#visited: avg = 543.1, max = 4889		
	#generated: avg = 936.6				#generated: avg = 3096.1		

■ **Table 5** Bucket Sampling for Klee–Minty cubes.

5.3 Crowd Sampling (Algorithm CS)

This strategy is inspired by the *stochastic enumeration* (SE) algorithm, which was proposed in 2013 by Reuven Y. Rubinstein [23] to count the number of nodes of a tree. The algorithm has a budget parameter B , imposing a limit on the number of nodes that we want to keep for further expansion. The algorithm consists of an alternating sequence of expansion and condensation phases. The main loop is as follows.

CS-2E [Expansion]

Expand every node of the current set A .

Let A' denote the resulting multi-set (or set, after identifying duplicates).

CS-2C [Condensation]

If $|A'| \leq B$, let $A := A'$.

Otherwise, randomly partition A' into B sets $S_1 \cup \dots \cup S_B$ of size $\lfloor \frac{|A'|}{B} \rfloor$ or $\lceil \frac{|A'|}{B} \rceil$. Condense each set S_i to a single element, and let A be the set of these B elements.

We call this method *Crowd Sampling*, because the samples are treated more like an unorganized crowd than a cooperating team.

Tables 6 and 7 show the results of this method. We also report the number of nodes that were saved by condensing multiple copies of the same node after the expansion step (“#saved”). Compared to our Bucket Sampling experiments (Tables 4–5), we see that the standard deviation is further reduced, at the expense of a larger number of visited nodes. The number of visited nodes can be controlled by adjusting the parameter B .

The purpose of Step CS-2C is to reduce the size of the active set A' to B . The method we have proposed is a straightforward way to achieve this, but one can think of many methods that are more sophisticated, taking into account the weights, for example.

5.4 Correctness of the Team Sampling scheme

We use the notation $\mathcal{P}(v \rightarrow w)$ for the sum of all path weights from v to w :

$$\mathcal{P}(v \rightarrow w) = \sum_{\text{paths } P \text{ from } v \text{ to } w} e(P)$$

Permutations						
m	$n = 8$			$n = 9$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$37,629.0 \pm 21,347.7$	7.31×10^4	0.19427	$335,864.8 \pm 5.66 \times 10^5$	1.93×10^6	0.57459
10^2	$41,642.1 \pm 31,007.1$	1.77×10^5	0.04256	$463,857.8 \pm 7.90 \times 10^5$	6.76×10^6	0.14564
10^3	$40,783.3 \pm 34,004.7$	2.51×10^5	0.00616	$385,896.2 \pm 7.23 \times 10^5$	1.26×10^7	0.03273
10^4	$40,390.6 \pm 36,143.8$	7.15×10^5	0.00177	$365,727.5 \pm 6.22 \times 10^5$	2.78×10^7	0.00760
10^5	$40,330.1 \pm 34,894.1$	8.50×10^5	0.00021	$366,894.1 \pm 5.96 \times 10^5$	2.89×10^7	0.00079
10^6	$40,336.7 \pm 35,324.7$	1.18×10^6	0.00003	$363,448.7 \pm 5.95 \times 10^5$	6.57×10^7	0.00018
	40,320 #visited: avg = 260.3, max = 266 #generated: avg = 893.2 #saved: avg = 189.6 = 21.2%			362,880 #visited: avg = 341.1, max = 348 #generated: avg = 1329.5 #saved: avg = 256.0 = 19.3%		

Table 6 Crowd Sampling for the permutation DAG with a budget $B = 10$.

Klee–Minty cubes						
m	$n = 10$			$n = 12$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$1,508.5 \pm 1,520.0$	4.91×10^3	0.32530	$2,741.1 \pm 2,785.5$	9.82×10^3	0.35839
10^2	939.6 ± 998.9	5.31×10^3	0.05647	$4,860.2 \pm 14,275.1$	1.38×10^5	0.28425
10^3	$1,028.3 \pm 1,292.8$	1.62×10^4	0.01578	$3,988.3 \pm 8,651.3$	1.38×10^5	0.03464
10^4	$1,001.7 \pm 1,342.2$	4.10×10^4	0.00409	$4,124.8 \pm 11,867.5$	6.98×10^5	0.01692
10^5	$1,018.4 \pm 1,442.2$	6.95×10^4	0.00068	$4,069.6 \pm 12,271.1$	1.07×10^6	0.00263
10^6	$1,024.6 \pm 1,483.7$	1.28×10^5	0.00012	$4,082.1 \pm 13,309.8$	3.93×10^6	0.00096
	1,024 #visited: avg = 709.3, max = 1441 #generated: avg = 2486.1 #saved: avg = 392.2 = 15.8%			4,096 #visited: avg = 937.7, max = 1839 #generated: avg = 3892.1 #saved: avg = 416.2 = 10.7%		

Table 7 Crowd Sampling for Klee–Minty cubes with a budget $B = 10$.

Lemma 7. Suppose that at some stage in Algorithm TS, we have a current estimate X^{current} and a current active set A with weights $W[v]$ for $v \in A$. If we perform only expansion steps (Step 2E) until the end of the algorithm, then the value of the estimate X at termination can be predicted deterministically, and it is

$$X^{\text{current}} + \sum_{v \in A} W[v] f_v, \quad (7)$$

for some factors f_v that depend only on the graph G , the costs $c(v)$, and the arc weights e_{vw} :

$$f_v = \sum_{w \in W} \mathcal{P}(v \rightarrow w) c(w) \quad (8)$$

In particular, an expansion step does not change the value of (7).

Proof. It is clear that the last sentence implies the whole lemma, since, at termination, $A = \emptyset$, and X^{current} is the final value of X .

Let $v_1, \dots, v_d$ be the $d = d^+(u)$ successors of u . Then f_u satisfies the relation

$$f_u = c(u) + \sum_{i=1}^d e_{uv_i} f_{v_i}. \quad (9)$$

Expanding vertex $u \in A$ adds $c(u)W[u]$ to X^{current} , and it adds $e_{uv_i}W[u]$ to each $W[v_i]$, so the expression (7) increases by $W[u](c(u) + \sum_{i=1}^d e_{uv_i} f_{v_i})$. On the other hand, u is removed from A , causing a decrease by $W[u]f_u$. By (9) these two changes cancel out. ◀

We now show that the value of (7) is also left invariant by a condensation step. In contrast to the expansion step (Lemma 7), this holds only in expectation:

► **Lemma 8.** *After a condensation step, the expected value of $\sum_{v \in A} W[v]f_v$ is the same as before.*

Proof. Condensing replaces a set S by one of its members $u \in S$, where u is chosen with probability $W[u] / \sum_{v \in S} W[v]$. Let $A' = A \setminus S \cup \{u\}$ be the new active set, and let W' denote the new weights. The new weight of u is $W'[u] = \sum_{v \in S} W[v]$. The weights in $A \setminus S$ are unchanged. Thus we get

$$E \left[\sum_{v \in A'} W'[v]f_v \right] = \sum_{v \in A \setminus S} W[v]f_v + \sum_{u \in S} \left(\Pr[u \text{ is chosen}] \times W'[u]f_u \right) \quad (10)$$

$$= \sum_{v \in A \setminus S} W[v]f_v + \sum_{u \in S} \left(\frac{W[u]}{\sum_{v \in S} W[v]} \times \sum_{v \in S} W[v]f_u \right) \quad (11)$$

$$= \sum_{v \in A \setminus S} W[v]f_v + \sum_{u \in S} W[u]f_u = \sum_{v \in A} W[v]f_v, \quad (12)$$

as claimed. ◀

► **Theorem 9.** *Algorithm TS produces an estimate with the same expectation as Algorithm P:*

$$E[X] = c_e(G) = \sum_{v \in V} \mathcal{P}(s_0 \rightarrow v) c(v)$$

Proof. Lemmas 7 and 8 together imply that, at any time during the algorithm,

$$E[X^{\text{final}}] = X^{\text{current}} + \sum_{v \in A} W[v]f_v.$$

If we apply this to the starting situation of Algorithm TS, with $A = \{s_0\}$, $W[s_0] = 1$, and $X^{\text{current}} = 0$, we get $E[X^{\text{final}}] = f_{s_0}$, which equals the claimed quantity. ◀

We mention that this analysis gives a combinatorial interpretation of the expectation of the random variables F_v introduced in Section 3 to compute the variance of the estimate X . Comparing (9) with (6) and taking note of the fact that μ_j in (6) stands for $E[F_{u_j}]$, we see that f_v and $E[F_v]$ satisfy the same recurrence relation. If v is a sink, (9) and (6) specify the same values $f_v = E[F_v] = c(v)$, which can be used as initial conditions. Thus:

► **Proposition 10.** *The expected value of F_v equals f_v .* ◀

5.5 Generalized Condensation

The Team Sampling algorithm shown above proposes a specific method for the Condensation Step (choice of probabilities and weight update). We will see that this is just one of many possibilities.

As mentioned, Crowd Sampling was modeled after the Stochastic Enumeration algorithm of Rubinstein [23], which works for trees. For a tree, the algorithm becomes very simple: In the expansion step, all backward proportions $\tilde{q}_{uv}$ are 1, and hence the children just get the same weights as their parents. If the resulting set A' of children is too big, it is simply replaced by a random sample of size B , and the weight is accordingly multiplied by $|A'|/B$. All nodes have the same weight.

Our Crowd Sampling method mimics the behavior of this algorithm only when $|A'|$ is a multiple of B . Otherwise, Crowd Sampling generates nodes of different weights, introducing irregularities into the process. In fact, for DAGs, nodes of different weights appear already for a different reason. For a set with weights, the meaning of “a random sample of size B ” is not obvious; see the survey [12] for two different potential interpretations. Let us take an engineering approach and formulate what is required to ensure that the estimate remains unbiased. It will turn out that this actually leaves a lot of freedom in performing the condensation step.

Assume that the set S contains k nodes with weights $W_1, W_2, \dots, W_k$. Suppose we decide that Step 2C should randomly select the i -th node with probability p_i , and assign weight W'_i to it in case it is selected. If we look at (10–12) in the proof of Lemma 8, we see that what we need for unbiasedness is the equation

$$p_1 W'_1 f_1 + \dots + p_k W'_k f_k = W_1 f_1 + \dots + W_k f_k. \quad (13)$$

Since we don't know the values f_i , this equation should hold for all $f_1, \dots, f_k$. Comparing coefficients leads to k equations

$$p_i W'_i = W_i, \quad (14)$$

for $i = 1, \dots, k$. Together with the equation $\sum_{i=1}^k p_i = 1$, this gives $k + 1$ equations for $2k$ unknowns p_i, W'_i , which is by far not enough to define a unique solution.³

The basic condensation step of Algorithm TS chooses $W'_i = \bar{W} := W_1 + \dots + W_k$ and $p_i = W_i/\bar{W}$, but it is not clear why this choice should be preferable over other choices. In fact, we can select arbitrary probabilities p_i (for example, uniform probabilities) and set $W'_i := W_i/p_i$. In this way, we can reinstate the forward proportions of Algorithm P, which Algorithm TS in its original form could not handle.

The generalized condensation step for Algorithm TS is thus as follows:

2C'. [Condense] Choose an arbitrary subset $S = \{u_1, u_2, \dots, u_k\}$ of A with $k \geq 2$ elements.

Choose positive probabilities p_i with $p_1 + p_2 + \dots + p_k = 1$.

Pick a random node u_i from S according to this distribution.

Replace S in A by the single element u_i , and set $W[u_i] := W[u_i]/p_i$.

³ We may even permit $\sum p_i < 1$, effectively throwing away the sample with probability $1 - \sum p_i$. In the single path variation (Algorithm P), the algorithm gets a chance to finish early and take a rest. This comes, of course, at the expense of a larger fluctuation of the estimate.

5.6 The variance for Team Sampling strategies

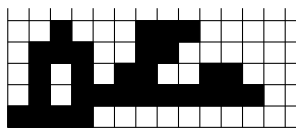
For particular strategies that specialize the Team Sampling procedure, it should in principle be possible to extend the results of Section 3 and compute the exact variance of the estimate X if the whole graph G with all forward and backward proportions is given. The general idea is that each condensation step contributes to the variance, because it replaces a set S by a single sample.

Knuth [19, Section 7.2.2.9, Theorem V] has given a method to compute the variance for Chen's version [9] of Bucket Sampling, where the heuristic function partitions the nodes into strata in an acyclic way. It would be interesting to get a similar analysis for Crowd Sampling.

5.7 Relation to dynamic programming

Many counting algorithms are based on *dynamic programming*, or the *transfer-matrix method* as it is sometimes called depending on the implementation. We show that these algorithms can be seen as a special case of Bucket Sampling.

We illustrate this with the problem of counting directed animals, following Conway [10]. A *directed animal* (see Figure 4) is a polyomino in the first quadrant such that every occupied cell, except the cell at the bottom-left corner, has an occupied cell to its left or at its bottom (or both). Directed animals are theoretically well-understood [13], and they are chosen in our example for illustration only.



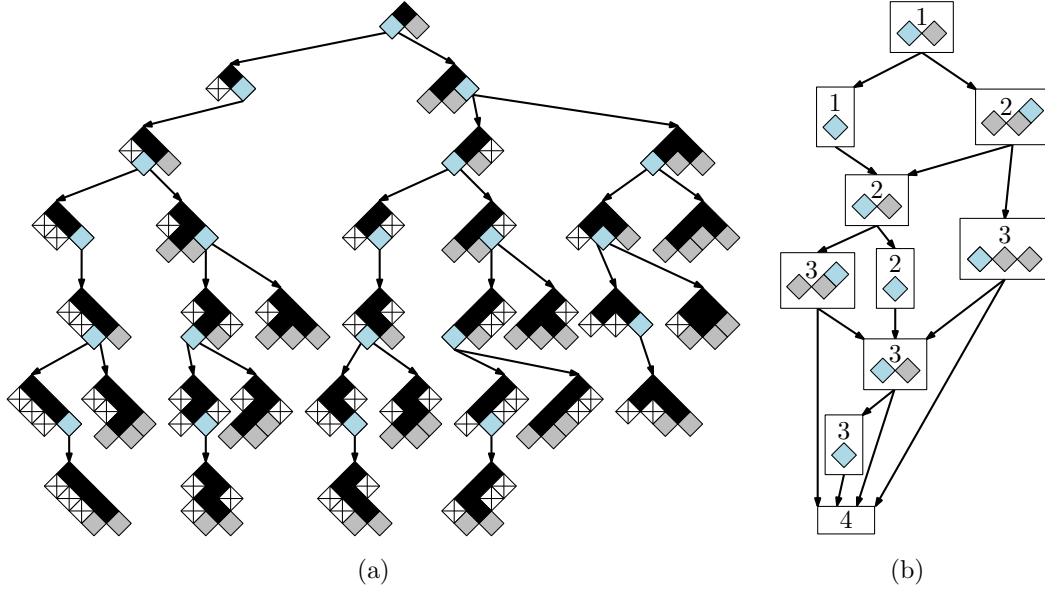
■ **Figure 4** A directed animal with 29 cells

Figure 5a shows an *enumeration tree* T for the directed animals. The animals have been turned by 135° and grow from top to bottom. The black cells are the cells that are already part of the animal, and the shaded cells are the adjacent cells below the chosen cells whose inclusion has not been decided yet. At each tree node, we take the leftmost undecided cell in the highest row (shaded in blue) and decide whether it is excluded (marked by a cross) or included (turned black). If there is only a single shaded cell, it has to be included. In this way, we try out all possibilities of generating a directed animal. We stop when we have 4 black cells, and hence the 13 leaves of this tree represent the directed animals of size 4.

The main idea of dynamic programming is that we don't need to remember the whole animal as it is built: For determining how to continue, it suffices to know the *pattern* of the grey cells and the *number* of black cells. Nodes for which this information agrees have equivalent subtrees. By merging such nodes, we arrive at the DAG G shown in Figure 5b. The directed animals correspond now to the source-sink paths in G .

The DAG G is smaller than the tree T from which it originated. (For larger examples than the one shown in the picture, the reduction in size is more impressive.) However, for the single-path algorithm (Algorithm P), it makes *absolutely no difference* whether it moves in the tree T to count leaves according to Theorem 1 or in the DAG G with update rule (U_1) to count source-sink paths according to Theorem 2. (In this spirit, Conway's reference to the DAG in [10, Fig. 2] as a "tree" is perhaps not completely off-track.)

We can use Bucket Sampling with a heuristic function h that maps the nodes of the tree T to the corresponding nodes in the DAG G . It turns out that, in this way, we arrive



622 **Figure 5** (a) an enumeration tree T for directed animals and (b) the associated dynamic-
 623 programming DAG G , adapted from Conway [10, Fig. 1 and 2]

637 essentially at the classic dynamic-programming algorithm for counting objects by the DAG:
 638 We scan the DAG top-down, adding the weight of each node to all its children. Since only
 639 nodes with equal subtrees are thrown together in one bucket, the algorithm gives the exact
 640 result.

641 The possibilities of Bucket Sampling, of course, go beyond this: We are allowed to merge
 642 nodes whose subtrees are different (but hopefully at least similar). The transition from the
 643 tree to the DAG, which is crucial for the speed-up in dynamic programming, is, however, not
 644 essential for Bucket Sampling, because it is always implicitly included. Perhaps, advantages
 645 of working with a DAG instead of an enumeration tree will show up in connection with other
 646 Team Sampling strategies.

647 **6 Conclusion**

648 We have proposed two very general methods for getting unbiased estimates for the size of a
 649 DAG. There is ample opportunity to come up with more ideas and try them out; we have
 650 barely scratched the surface with our experiments. For example, one can combine the ideas
 651 of Bucket Sampling and Crowd Sampling, allowing up to B nodes to occupy one bucket.

652 For the choice of forward proportions $\vec{p}_{uv}$, Importance Sampling gives some guidelines
 653 about what one wants to achieve (see Section 4). By contrast, the role of the backward
 654 proportions $\tilde{q}_{uv}$ is not understood at all. One may even allow them to become zero: $\tilde{q}_{uv} = 0$
 655 effectively means that the sample is ignored when the algorithm traverses the arc uv .

656 **6.1 Provable approximation quality?**

661 Vaisman and Kroese showed that Stochastic Enumeration is a fully polynomial randomized
 662 approximation scheme (FPRAS) for counting the number of nodes of trees, for *most* trees in

a certain (rather restricted) class of random trees [26, Theorem 4.3⁴]. It would be interesting to see whether Path Sampling or Team Sampling leads to an FPRAS for some combinatorial counting problems. We mention a few such problems below.

6.2 Extension to arc weights

The extension to estimating the *sum* of all arc weights in an arc-weighted graph is straightforward: When proceeding from u to v , the algorithm adds $W \cdot d^+(u) \cdot c(uv)$ to X , where $c(uv)$ is some cost function associated to the arc uv . Conceptually, we could simply subdivide every arc and let the newly inserted nodes take over the role of the arcs.

6.3 Some applications

6.3.1 Vertices of a polytope

For a polytope, a linear objective function induces an orientation of the graph of the polytope: It directs each edge in the direction of improvement of the objective function, provided that the objective function is sufficiently generic and does not assign the same value to adjacent vertices.

Avis and Devroye have used the tree version of Algorithm P to estimate the number of vertices of a polytope [3]. The simplex algorithm generates a path from each vertex of a polytope to the optimum vertex, and thereby defines a spanning tree of the vertices. This spanning tree is at the core of the *reverse search* method for enumerating all vertices, and it is the tree to which Avis and Devroye apply Algorithm P. Salomone, Vaisman, and Kroese [24] have applied the Stochastic Enumeration method (or Crowd Sampling) to this tree.

However, the graph of a polytope whose edges are oriented by an objective function is naturally a DAG. We intend to compare our approach against the tree-based estimation methods of [3] and [24].^{5,6}

6.3.2 Polyominoes

Various type of polyominoes have been counted by the *transfer-matrix* method, which is a dynamic-programming algorithm for counting that avoids to enumerate the polyominoes individually, see [5, 6] for recent advances, and [10] for an overview that includes also other problems. As discussed in Section 5.7, many dynamic-programming algorithms can be interpreted as accumulating (possibly weighted) source-sink paths in a DAG. These algorithms are run on huge DAGs to get exact counts, exhausting limits of computer time and memory consumption. With our approach, the same ideas can be leveraged to get

⁴ In the statement of this theorem, the budget parameter B for the algorithm is not specified. It must be set to kB' , where B' is defined in [26, Theorem 4.1, p. 52], depending polynomially on the tree height and the maximum degree k , as well as the parameters of the random tree model (Radislav Vaisman, personal communication, July 2025).

⁵ Our discussion ignores an important technicality: The objects that the simplex algorithm visits are actually the *feasible bases*, and for polytopes that are not simple, a vertex can correspond to several bases. The standard way to treat such *degenerate* or nonsimple polytopes, where more than n facets meet in a vertex, is to perturb the constraints, resulting in a simple polytope, and count only one of the many vertices that correspond to a vertex of the original polytope. See [3] for details.

⁶ We mention that the permutation DAG is also obtainable as the graph of a polytope, namely the *permutahedron*: The vertex coordinates of the permutahedron are given by the inverse permutations of the permutations that form the nodes of the permutation DAG as described here. For any generic objective function, the directed graph of the permutahedron edges is isomorphic to the permutation DAG.

estimates far beyond the polyomino sizes for which exact computation is feasible. This might be an alternative to Markov-chain based sampling techniques [25]. With appropriate weights $c(v)$, one can also gather statistics about various properties of polyominoes.

6.3.3 Triangulations of a planar point set

Alvarez and Seidel [2] have given an algorithm with runtime $O(2^n n^2)$ for counting the triangulations of an n -point set in the plane. They model triangulations as source-sink paths in a DAG with $n2^{n-3}$ nodes. Our algorithms can therefore be readily applied. The quality of the resulting approximation remains to be explored. The state of the art about approximate counting of triangulations is rather weak: There is only a subexponential-time approximate counting method with a subexponential multiplicative error [1]. The current fastest algorithm for exact counting takes time $n^{O(\sqrt{n})}$ [20].

6.3.4 Linear extensions of a two-dimensional partial order

If we start from an arbitrary permutation $\pi = (\pi_1, \dots, \pi_n)$ in the permutation DAG, the reachable nodes are the linear extensions of a two-dimensional partial order $\prec$, which is defined by the rule $i \prec j \Leftrightarrow i < j \wedge \pi_i \leq \pi_j$. Thus our path sampling methods can be used to estimate the number of linear extensions of such a partial order. The computation of indegrees must be adjusted to count only those predecessor permutations that can be reached from π .

Counting the linear extensions is #P-complete, even for a two-dimensional partial order, as proved by Dittmer and Pak [11, Theorem 1.3].⁷ In any case, there already exists a fully polynomial randomized approximation scheme for the number of linear extensions of an arbitrary poset, because the number of linear extensions is directly related to the volume of the order polytope, and the polytope volume can be approximated by Markov chain Monte Carlo methods, see for example [7, Section 3] for details and references. Our path sampling methods might lead to alternative approximation schemes that are simpler.

Acknowledgment. The ideas of this paper were inspired by discussions initiated at the workshop on *Computational Polyhedral Geometry*, which took place on June 26, 2025 as part of the Computational Geometry Week (CG Week) in Kanazawa, Japan.⁸ I thank Laura Casabella and Dante Luber for organizing this workshop, and the other participants for the lively discussions.

References

- 1 Victor Alvarez, Karl Bringmann, Saurabh Ray, and Raimund Seidel. Counting triangulations and other crossing-free structures approximately. *Computational Geometry*, 48(5):386–397, 2015. Special Issue on the 25th Canadian Conference on Computational Geometry (CCCG). doi:10.1016/j.comgeo.2014.12.006.
- 2 Victor Alvarez and Raimund Seidel. A simple aggregative algorithm for counting triangulations of planar point sets and related problems. In *Proceedings of the 29th Annual Symposium on Computational Geometry (SoCG'13)*, pages 1–8, New York, NY, USA, 2013. Association for Computing Machinery. doi:10.1145/2462356.2462392.

⁷ There is a subsequent journal article by the same authors with the same title, which, however, omits this result and does not even mention it.

⁸ <https://socg25.github.io/workshops.html>

- 745 **3** David Avis and Luc Devroye. Estimating the number of vertices of a polyhedron. *Information*
746 *Processing Letters*, 73(3):137–143, 2000. doi:10.1016/S0020-0190(00)00011-9.
- 747 **4** József Balogh and Robin Pemantle. The Klee–Minty random edge chain moves with linear
748 speed. *Random Structures & Algorithms*, 30(4):464–483, 2007. doi:10.1002/rsa.20127.
- 749 **5** Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. In *Proceedings of*
750 *the 2024 Symposium on Algorithm Engineering and Experiments (ALENEX)*, pages 133–143.
751 SIAM, 2024. doi:10.1137/1.9781611977929.10.
- 752 **6** Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. Preprint, 2025. Submit-
753 ted. doi:10.21203/rs.3.rs-4304962/v1.
- 754 **7** Graham Brightwell and Peter Winkler. Counting linear extensions. *Order*, 8:225–242, 1991.
755 doi:10.1007/BF00383444.
- 756 **8** Sourav Chatterjee and Persi Diaconis. The sample size required in importance sampling. *Ann.*
757 *Appl. Probab.*, 28(2):1099–1135, 2018. doi:10.1214/17-AAP1326.
- 758 **9** Pang C. Chen. Heuristic sampling: A method for predicting the performance of tree searching
759 programs. *SIAM Journal on Computing*, 21(2):295–315, 1992. doi:10.1137/0221022.
- 760 **10** Andrew R. Conway. The design of efficient dynamic programming and transfer matrix
761 enumeration algorithms. *Journal of Physics A: Mathematical and Theoretical*, 50(35):353001
762 (28 pp.), aug 2017. arXiv:1610.09806, doi:10.1088/1751-8121/aa8120.
- 763 **11** Samuel Dittmer and Igor Pak. Counting linear extensions of restricted posets, 2018. arXiv:
764 1802.06312.
- 765 **12** Pavlos S. Efraimidis. Weighted random sampling over data streams. In Christos Zaroliagis,
766 Grammati Pantziou, and Spyros Koutogiannis, editors, *Algorithms, Probability, Networks, and*
767 *Games: Scientific Papers and Essays Dedicated to Paul G. Spirakis on the Occasion of His 60th*
768 *Birthday*, pages 183–195. Springer International Publishing, Cham, 2015. arXiv:1012.0256,
769 doi:10.1007/978-3-319-24024-4_12.
- 770 **13** D. Gouyou-Beauchamps and G. Viennot. Equivalence of the two-dimensional directed animal
771 problem to a one-dimensional path problem. *Advances in Applied Mathematics*, 9(3):334–357,
772 1988. doi:10.1016/0196-8858(88)90017-6.
- 773 **14** Bernd Gärtner, Martin Henk, and Günter M. Ziegler. Randomized simplex algorithms on
774 Klee–Minty cubes. *Combinatorica*, 18(3):349–372, 1998. doi:10.1007/PL00009827.
- 775 **15** Marshall Hall and D. E. Knuth. Combinatorial analysis and computers. *The American*
776 *Mathematical Monthly*, 72(2):21–28, 1965. doi:10.2307/2313307.
- 777 **16** Victor Klee and G. J. Minty. How good is the simplex algorithm? In O. Shisha, editor,
778 *Inequalities III*, pages 159–175. Academic Press, New York, 1972.
- 779 **17** Donald E. Knuth. Estimating the efficiency of backtrack programs. *Mathematics of*
780 *Computation*, 29:122–136, 1975. URL: [https://www.ams.org/journals/mcom/1975-29-129/](https://www.ams.org/journals/mcom/1975-29-129/S0025-5718-1975-0373371-6/)
781 [S0025-5718-1975-0373371-6/](https://www.ams.org/journals/mcom/1975-29-129/S0025-5718-1975-0373371-6/).
- 782 **18** Donald E. Knuth. *Combinatorial Algorithms, Part 2*, volume 4B of *The Art of Computer*
783 *Programming*. Addison-Wesley, 2023.
- 784 **19** Donald E. Knuth. *Combinatorial Algorithms, Part 3*, volume 4C of *The Art of Computer*
785 *Programming*. Addison-Wesley, 2027+. In preparation. Draft of Section 7.2.2.9, “Estimating
786 Backtrack Costs” at <https://cs.stanford.edu/~knuth/fasc9c.ps.gz>, version March 27,
787 2022.
- 788 **20** Dániel Marx and Tillmann Miltzow. Peeling and nibbling the cactus: Subexponential-time
789 algorithms for counting triangulations and related problems. In Sándor P. Fekete and Anna
790 Lubiwi, editors, *32nd International Symposium on Computational Geometry, SoCG 2016,*
791 *Boston, MA, USA, June 14-18, 2016*, LIPIcs, pages 52:1–52:16. Schloss Dagstuhl - Leibniz-
792 Zentrum für Informatik, 2016. doi:10.4230/LIPICS.SOCG.2016.52.
- 793 **21** Paul W. Purdom. Tree size by partial backtracking. *SIAM Journal on Computing*, 7(4):481–491,
794 1978. doi:10.1137/0207038.

- 795 22 Günter Rote. NumPSLA – an experimental research tool for pseudoline arrangements and
 796 order types. In Jan Kratochvíl and Giuseppe Liotta, editors, *41st European Workshop on*
 797 *Computational Geometry (EuroCG 2025)*, pages 18:1–18:8, April 2025. [arXiv:2503.02336](#).
- 798 23 Reuven Rubinfeld. Stochastic enumeration method for counting NP-hard problems.
 799 *Methodology and Computing in Applied Probability*, 15:249–291, 2013. [doi:10.1007/s11009-011-9242-y](#).
- 800 24 Robert Salomone, Radislav Vaisman, and Dirk Kroese. Estimating the number of vertices in
 801 convex polytopes. In *Proceedings of the 4th Annual International Conference on Operations*
 802 *Research and Statistics (ORS 2016), Singapore, 2016*, pages 96–105. Global Science and
 803 Technology Forum, Singapore, 2016. [doi:10.5176/2251-1938_OR16.25](#).
- 804 25 D. Stauffer. Monte Carlo study of density profile, radius, and perimeter for percolation clusters
 805 and lattice animals. *Phys. Rev. Lett.*, 41:1333–1336, Nov 1978. [doi:10.1103/PhysRevLett.41.1333](#).
- 806 26 Radislav Vaisman and Dirk P. Kroese. Stochastic enumeration method for counting
 807 trees. *Methodology and Computing in Applied Probability*, 19:31–73, 2017. [doi:10.1007/s11009-015-9457-4](#).

811 A Importance Sampling for Klee-Minty cubes

812 As mentioned in Section 4, we have tried Importance Sampling by outdegree also for Klee-
 813 Minty cubes. In comparison to Table 2, the results of Table 8 show a substantial improvement
 814 in the theoretical standard deviation, by several orders of magnitude. Like for permutations,
 815 the comparison in terms empirical behavior is not at all clear. Even with 10 million samples,
 816 the fluctuations are too high to make a reasonable assessment.

Klee-Minty cubes						
m	$n = 10$			$n = 12$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	87.5 ± 102.1	3.34×10^2	0.38153	$2,028.6 \pm 3,386.0$	1.02×10^4	0.50207
10^2	358.4 ± 995.7	7.46×10^3	0.20824	$2,533.0 \pm 13,646.2$	1.33×10^5	0.52576
10^3	$649.8 \pm 3,521.7$	9.34×10^4	0.14378	$1,699.2 \pm 9,894.0$	1.79×10^5	0.10549
10^4	$904.4 \pm 24,382.4$	2.38×10^6	0.26310	$2,084.0 \pm 30,840.8$	2.62×10^6	0.12584
10^5	$863.3 \pm 15,913.2$	2.91×10^6	0.03368	$2,686.7 \pm 62,038.0$	1.05×10^7	0.03895
10^6	$963.2 \pm 33,681.1$	1.74×10^7	0.01808	$4,501.2 \pm 8.53 \times 10^5$	6.49×10^8	0.14407
10^7	$986.2 \pm 58,292.5$	9.65×10^7	0.00979	$4,392.0 \pm 1.20 \times 10^6$	3.00×10^9	0.06841
	$1,024 \pm 2.141 \times 10^6$			$4,096 \pm 7.762 \times 10^{10}$		
	#visited: avg = 22.4, max = 80			#visited: avg = 29.2, max = 104		

828 ■ **Table 8** Algorithm P for Klee-Minty cubes with Importance Sampling by outdegree

829 A.1 Importance Sampling by the size of the reachable set

830 As mentioned, when Path Sampling is applied to a rooted tree, the ideal forward proportions
 831 are known: The probabilities $\vec{p}_{uv}$ should be proportional to the cost of the subtrees rooted at
 832 the children v of node u . For DAGs, it is not clear whether such a perfect choice of forward
 833 proportions that completely eliminates variations of X exists. As a substitute for the size of
 834 the subtree rooted at v , we may take the size R_v of the node set reachable from v .

835 For Klee-Minty cubes, we are in the lucky situation that the size R_v of the reachable set
 836 is readily available: The Klee-Minty cube of dimension n has a directed Hamilton path, in

which the last coordinate x_n increases in an arithmetic progression $0, 1, 2, \dots, 2^n - 1$, see Figure 3. Thus, $R_v = 2^n - x_n(v)$.

We made two experiments where R_v is taken into account for computing the forward proportions. We emphasize that this is not suitable in practical applications: If the size of the reachable set were available, there would be no need to compute an estimate by running Algorithm P. However, the experiment teaches us what we might be looking for in the choice of forward proportions.

In the first experiment (Table 9), we simply chose the probabilities $\vec{p}_{uv}$ for the successors v of a node u proportional to R_v . We see no improvement over simply using the outdegree.

In the second experiment (Table 10), we took into account the fact that a node v with large indegree is visited from several predecessors. We therefore chose the probabilities proportional to $R(v)/d^-(v)$. Here we get much better estimates, and we see a substantial reduction of the standard deviation. While these weights are not perfect, they indicate the direction in which one should look. Conceivably, a perfect choice of parameters can be obtained by adjusting also the backward proportions $\vec{q}_{uv}$.

Klee–Minty cubes						
m	$n = 10$			$n = 12$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	202.2 ± 217.4	6.97×10^2	0.34488	$994.4 \pm 1,564.7$	4.56×10^3	0.45878
10^2	$473.9 \pm 1,054.7$	9.21×10^3	0.19432	$1,171.3 \pm 3,682.9$	3.44×10^4	0.29384
10^3	$976.3 \pm 10,248.4$	2.90×10^5	0.29717	$2,154.3 \pm 16,630.3$	4.04×10^5	0.18755
10^4	$1,121.8 \pm 30,028.1$	2.82×10^6	0.25136	$2,337.7 \pm 30,091.7$	2.26×10^6	0.09668
10^5	$897.2 \pm 13,521.5$	2.82×10^6	0.03143	$3,107.8 \pm 98,543.9$	2.37×10^7	0.07613
10^6	$1,026.2 \pm 61,065.3$	4.95×10^7	0.04828	$3,255.6 \pm 1.65 \times 10^5$	7.04×10^7	0.02163
10^7	$1,006.2 \pm 65,644.4$	1.29×10^8	0.01282	$3,635.2 \pm 1.01 \times 10^6$	3.11×10^9	0.08555
	$1,024 \pm 2.080 \times 10^6$			$4,096 \pm 6.025 \times 10^{11}$		
	#visited: avg = 22.4, max = 80			#visited: avg = 29.2, max = 112		

Table 9 Algorithm P for Klee–Minty cubes with Importance Sampling by the size of the reachable set

Klee–Minty cubes						
m	$n = 10$			$n = 12$		
	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	583.2 ± 611.6	2.07×10^3	0.35412	$2,778.3 \pm 5,045.8$	1.67×10^4	0.60117
10^2	$1,299.8 \pm 4,881.0$	4.77×10^4	0.36690	$2,231.1 \pm 5,569.1$	4.60×10^4	0.20638
10^3	$897.7 \pm 2,388.1$	4.77×10^4	0.05313	$4,351.1 \pm 55,366.3$	1.74×10^6	0.39894
10^4	$976.6 \pm 3,751.7$	1.52×10^5	0.01551	$3,611.7 \pm 46,923.6$	4.00×10^6	0.11076
10^5	$1,066.5 \pm 19,675.0$	5.92×10^6	0.05552	$3,851.3 \pm 58,920.5$	1.00×10^7	0.02597
10^6	$1,034.9 \pm 12,622.9$	6.65×10^6	0.00643	$4,031.7 \pm 1.21 \times 10^5$	7.25×10^7	0.01799
10^7	$1,023.2 \pm 8,554.4$	9.69×10^6	0.00095	$4,043.8 \pm 1.38 \times 10^5$	2.93×10^8	0.00725
	$1,024 \pm 1.071 \times 10^4$			$4,096 \pm 9.419 \times 10^5$		
	#visited: avg = 42.5, max = 112			#visited: avg = 58.7, max = 160		

Table 10 Algorithm P for Klee–Minty cubes with Importance Sampling by the size of the reachable set divided by the indegree

B Details about implementing Klee-Minty cubes

The natural way to associate a bit vector $(y_1, \dots, y_n)$ to the vertices of a Klee-Minty cube (4) (Section 2.2) is derived from the standard cube: $y_i = 1$ if x_i is at the ceiling, i.e., $x_i = 2^i - 1 - x_{i-1}$, and $y_i = 0$ if x_i is at the floor, i.e., $x_i = x_{i-1}$. The n neighbors of $(y_1, \dots, y_n)$ are obtained by simply flipping a single bit. The downside of this representation is that it is not easy to see whether an edge is improving.

Our program uses a different representation $(z_1, \dots, z_n)$, which was introduced by Gärtner, Henk, and Ziegler [14]:

$$\begin{aligned} z_n &= y_n \\ z_{n-1} &= y_n \oplus y_{n-1} \\ z_{n-2} &= y_n \oplus y_{n-1} \oplus y_{n-2} \\ &\vdots \\ z_1 &= y_n \oplus y_{n-1} \oplus y_{n-2} \oplus \dots \oplus y_2 \oplus y_1 \end{aligned}$$

It is the sequence of partial sums of the y_i 's using addition modulo 2, or the exclusive-or operation $\oplus$.

The outgoing arcs are identified by the zero bits in this representation. If we follow such an arc, we must flip the i -th bit together with all lower-order bits. The reason is that while the orientation of all incident edges with higher index is maintained, the lower-index directions are perverted.

More concretely, we represent a vertex by the n -bit integer $(z_n z_{n-1} \dots z_2 z_1)_2$. It turns out that the value of this integer is equal to the x_n -coordinate of the vertex.

C Purdom's partial backtracking method

Purdom [21], in 1978, suggested the following variation of Knuth's Path Sampling algorithm for trees: Instead of proceeding to a single child, proceed to at most m randomly selected samples among the children. The parameter m can itself be chosen at random. In particular, Purdom looked at the setting of binary trees, and he proposed to choose $m = 2$ or $m = 1$ with probability p and $1 - p$ respectively.

Purdom analyzed analytically the number of visited nodes as well as the variance of the estimated tree size, for a certain class of random binary trees. He also made experiments with a backtracking program.

This method could be easily adapted to DAGs, but it will probably not compete with the more sophisticated Team Sampling methods, which allow a more direct control of the number of visited nodes. Still we want to cite one sentence from the conclusion of the paper: "The analysis suggests that good results can be obtained by choosing the number of branches to investigate so that each level, from the root down to the bulk of the nodes, is examined at about the same frequency." This is in line with the strategy of the Crowd Sampling scheme, which sets a fixed budget B . In case of a tree, Crowd Sampling would indeed look at up to B nodes in each level.